

Core idea first.

What is Entire?

Think of **Git for AI-agent work context**.

Normal Git remembers:

```
code changes
commits
branches
```

But when coding agent changes code, Git does **not** remember:

```
User prompt
Agent response
Why agent made change
Files agent inspected
Tools agent used
Commands agent ran
Agent's intermediate work
```

Entire captures this context and connects it to Git history. It creates checkpoints containing session metadata separately from your normal code branch.  GitHub +1

Example:

```
You:
"Add JWT authentication"

      ↓

Coding Agent

      ↓

reads:
auth/User.java
auth/AuthService.java

      ↓

edits:
AuthService.java
SecurityConfig.java

      ↓

runs:
```

Caveat: `./gradlew test`

↓

Git commit

↓

Entire checkpoint

Later:

Bash

`entire explain`

You can understand **why code changed**, not merely what changed.

What hackathon topic means

Your topic:

Bring Entire to a New Agent or Workflow

means:

Take an AI coding environment that Entire does not properly support yet, and build an integration/adaptor so Entire can capture its development context.

This is important:

A thin wrapper that merely runs an `entire` command is not enough.

Your implementation needs to live in the designated **Entire fork** and meaningfully capture/use agent context.

What "External Agent" means

Imagine Entire currently understands agents like:

```
Claude Code
Codex
Cursor
Gemini CLI
Copilot CLI
OpenCode
...
```

Each agent has different ways of exposing events.

For example, Entire's current integrations use agent-specific hooks/plugins:

```
Claude Code → .claude/settings.json
Codex       → .codex/hooks.json
Cursor      → .cursor/hooks.json
Copilot CLI → .github/hooks/entire.json
Gemini CLI  → .gemini/settings.json
OpenCode    → .opencode/plugins/entire.ts
```

Entire then converts those events into its common session/checkpoint model.  [GitHub](#)

Your job is essentially:

```
NEW AGENT / WORKFLOW
  |
  ▼
YOUR ADAPTER
  |
  ▼
Entire format
  |
  ▼
Sessions / Checkpoints / Git
```

What should we build?

Before coding, we should choose **one target**.

Good targets:

1. New coding agent

Example:

```
Devin
Kilo Code
```

Roo Code
Windsurf
Zed Agent
Aider
OpenHands

Build:

entire-agent-<name>

that translates that agent's lifecycle/events into Entire's expected events.

This is probably strongest hackathon direction.

2. New editor

Example:

Zed
VS Code extension
JetBrains
Neovim

Agent works inside editor:

User
↓
Editor
↓
AI Agent
↓
Code changes

Your integration captures:

session start
prompt
agent response
tool calls
files changed
commits
session end

3. New workflow

This is more interesting.

Example:

```
GitHub Actions
  ↓
AI coding agent
  ↓
PR
  ↓
review
  ↓
fix
  ↓
commit
```

Make Entire understand this lifecycle.

Could capture:

```
Issue
  ↓
Agent prompt
  ↓
Agent work
  ↓
PR
  ↓
Review
  ↓
Fixes
  ↓
Final commit
```

What I would recommend

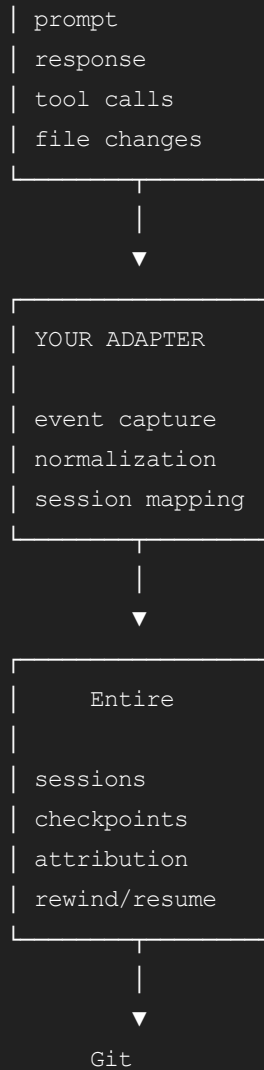
For hackathon:

Don't start by building a giant new AI agent.

Build an **Entire adapter** for an existing agent/workflow.

Architecture:

```
graph TD
    subgraph AI_Agent [AI Agent]
        direction TB
        A[ ]
    end
```



First thing we should do

Clone Entire's external-agent repository and study **one existing integration**.

Entire external-agents repository

Do not start coding yet.

First understand:

```
external-agents/  
├─ agents/  
│   ├── ...  
│   └─ entire-agent-xxx/  
│       ...  
└─ ...
```

Pick one existing adapter and trace:

```
Agent event
  ↓
adapter
  ↓
Entire event
  ↓
session
  ↓
checkpoint
  ↓
Git
```

There is already useful precedent. For example, an external Zed integration captures session/turn lifecycle and transcript information, while a Devin integration handles checkpointing, attribution, transcript analysis, and token accounting.  [GitHub](#) +1

Our development plan

I would do this in stages:

Phase 1 — Understand Entire

Learn:

```
Session
Checkpoint
Turn
Lifecycle events
Transcript
Attribution
Git integration
```

Phase 2 — Pick target

Example:

```
Zed Agent
      OR
OpenHands
      OR
Aider
      OR
some workflow
```

Phase 3 — Study target's API/events

Find:

```
How prompt is exposed?
How response is exposed?
How tool calls are exposed?
How file changes are exposed?
How session ID is obtained?
How session ends?
```

Phase 4 — Build adapter

```
Target Agent
  ↓
event listener
  ↓
normalize
  ↓
Entire lifecycle
```

Phase 5 — Test

We need to prove:

```
prompt captured
  ✓

agent response captured
  ✓

files attributed
  ✓

tool activity captured
  ✓

commit linked
  ✓

checkpoint created
  ✓

resume/rewind works
  ✓
```

Phase 6 — Demo

This is where hackathon points come.

Demo:

1. Start agent
2. Give task
3. Agent modifies code
4. Commit
5. Entire captures everything
6. Open checkpoint
7. Show:
 "Why was this code written?"
8. Rewind/resume

That gives judges a very clear **before** → **after** story.

One important insight

Don't think:

"We need to add another AI agent to Entire."

Think:

"We need to make Entire understand another source of AI-development context."

That distinction matters.

The real product is:

```
AI coding happens anywhere
      ↓
Entire captures context
      ↓
context becomes attached to Git
      ↓
development becomes explainable,
recoverable and traceable
```

That's our north star.

Next step: I suggest we inspect Entire's `external-agents` repo together, understand its architecture, then pick the easiest/strongest agent to integrate.

[stay in caveman mode — LITE]

so do we have to make a new coding agent like cursor and all but with it should have entire configured so when we change the code git also remember it so next time we dont have to give the context or codes changed

Yes — **but not exactly.**

You do not need to build a whole new Cursor-like coding agent.

Your hackathon goal can be:

```
Existing coding agent
  ↓
Your integration
  ↓
Entire
  ↓
Git + development context
```

What Entire adds

Normal Git remembers:

```
commit A
  ↓
changed files
  ↓
commit B
```

But Entire remembers richer context:

```
User prompt
  ↓
Agent reasoning/context
  ↓
Tool calls
  ↓
Files changed
```

↓
Git commit
↓
Entire checkpoint

So next time agent starts work, it can potentially access:

"What did we do before?"
"Why was this file changed?"
"What decisions did previous agent make?"
"What commands were run?"
"What was previous task?"

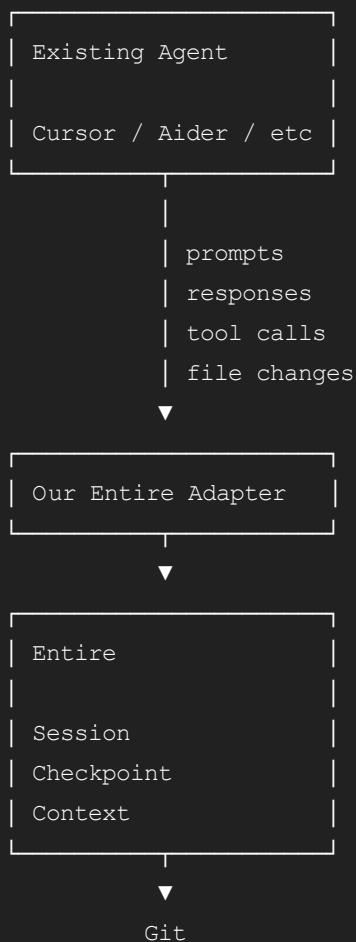
Instead of user saying:

"Here is context...
Here are files...
Previously we changed X...
Then we discovered Y..."

agent can retrieve that development history.

For hackathon

We could build something like:



Then demo:

Day 1

User: Build authentication

Agent → changes 8 files

Entire → records context + checkpoint

Git → commit

Day 2

User:

Continue authentication work.

Agent:

Entire context:

Previous task: authentication

Changed files: ...

Previous decisions: ...

Tests: ...

Last checkpoint: ...

Agent continues without user dumping old context.

One correction

Don't say:

"Git remembers everything."

Better:

Entire uses Git history plus AI-development context to preserve why/how changes happened.

That is core value.

For hackathon, I would **not build Cursor from scratch**. Waste of time.

Build **one strong integration into an existing agent/workflow**, then make context retrieval impressive.

Next, we should choose **which agent gives us easiest path + strongest demo**, then inspect its hooks/API and start coding.

[stay in caveman mode — LITE]

for the project repo name for hackathon what should we name it

Need name that sounds technical, clear, memorable.

My top picks:

1. **ContextForge** — best overall. Forge agent context into Git history.
2. **GitMind** — simple, memorable. Git + agent memory.
3. **ContextGit** — very clear, less flashy.
4. **AgentTrace** — focuses on agent activity/history.
5. **RecallCode** — agent remembers previous coding work.
6. **CodeMemory** — obvious product story.
7. **ContextFlow** — context follows development workflow.
8. **AgentVault** — stores valuable agent context.
9. **GitRecall** — retrieve previous development context.
10. **TraceForge** — technical, hackathon-friendly.

For hackathon repo, I'd pick:

ContextForge

Repo:

contextforge

Tagline:

Persistent memory for AI coding agents, powered by Entire.

Strong demo story:

Agent forgets → ContextForge remembers

If project is specifically an **Entire integration**, another strong name:

EntireBridge

But **ContextForge** sounds more like something you built, rather than simply a connector.

[stay in caveman mode — LITE]

so we are team of three so divide the work

Yes. Team of 3 should split by **core integration, context/memory, and product/demo**.
Avoid everyone touching same code.

Team split

Person 1 — Entire Integration Lead

Own:

Agent → Entire → Git

Tasks:

- Study existing Entire external-agent integrations.
- Understand Entire event/checkpoint architecture.
- Build adapter for chosen agent.
- Capture:
 - session start/end
 - prompts
 - responses
 - tool calls
 - file changes
 - Git commits
- Make checkpoint creation work reliably.
- Write integration tests.

Deliverable: Agent works with Entire end-to-end.

Person 2 — Context + Memory Lead

Own:

Entire history
↓
Context retrieval
↓
Agent

Tasks:

- Understand how previous Entire checkpoints/context can be retrieved.
- Design context format.
- Build mechanism to give previous context back to agent.
- Handle:
 - previous tasks
 - changed files
 - decisions
 - commands
 - relevant checkpoints
- Prevent huge context dumps.
- Test continuation across multiple sessions.

Deliverable: Agent can start new session and recover useful previous context.

This person builds your **"agent remembers"** feature.

Person 3 — Product + Demo + Testing Lead

Own:

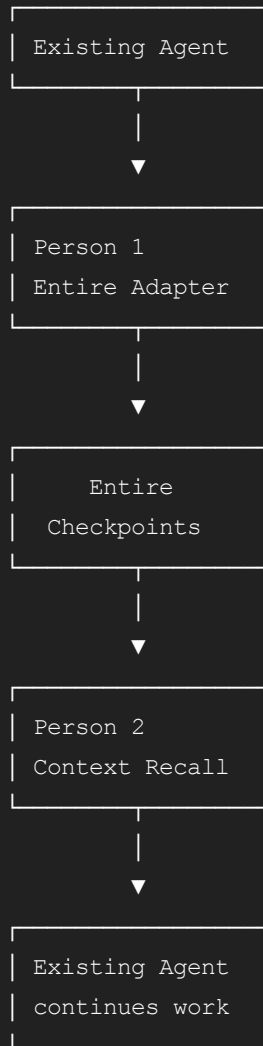
```
User experience
      +
Demo
      +
Quality
```

Tasks:

- Build simple CLI/UI around integration if needed.
- Create setup/install flow.
- Create sample repository.
- Build demo scenario.
- Test full workflow.
- Write README.
- Prepare architecture diagram.
- Prepare hackathon presentation/demo script.
- Find failure cases and bugs.

Deliverable: Judge can clone repo and understand/run project quickly.

How work connects



Person 3 wraps everything:
README + UX + tests + demo

Git workflow

Use separate branches:

```
main
|
├─ integration
|
├─ context-memory
|
└─ demo-ux
```

Person 1 owns integration branch.

Person 2 owns context branch.

Person 3 owns demo/docs/test branch.

Merge into `main` through PRs.

First-day plan

Don't immediately code.

Hour 1–2: All three understand Entire architecture together.

Then:

```
Person 1 → inspect existing integrations
Person 2 → inspect checkpoint/context retrieval
Person 3 → build demo repo + document expected workflow
```

After that: Person 1 gets basic integration working first. Person 2 plugs memory into it. Person 3 continuously tests and documents.

Most important: **agree on one agent target before splitting further**. Otherwise team can build three disconnected pieces.

[stay in caveman mode — LITE]

ok lets start with the folder structure first

Yes. Start simple. Don't over-engineer.

Assuming repo name **ContextForge**, I recommend:

```
contextforge/
├─ README.md
├─ LICENSE
├─ CONTRIBUTING.md
├─ .gitignore
├─
├─ docs/
│   ├─ architecture.md
│   ├─ setup.md
│   └─ demo.md
├─
├─ src/
│   └─ adapter/
│       └─ index.ts
```

```

├── events.ts
├── session.ts
├── context/
│   ├── index.ts
│   ├── collector.ts
│   ├── formatter.ts
│   └── recall.ts
├── entire/
│   ├── index.ts
│   ├── checkpoint.ts
│   └── client.ts
├── git/
│   ├── index.ts
│   └── history.ts
├── cli/
│   ├── index.ts
│   └── commands.ts
├── tests/
│   ├── adapter/
│   ├── context/
│   ├── entire/
│   └── integration/
├── examples/
│   └── demo-project/
├── scripts/
│   ├── setup.sh
│   └── demo.sh
├── package.json
├── tsconfig.json
└── .env.example

```

What each part does

adapter/

Agent-specific code.

```

Agent
  ↓
events
  ↓
adapter

```

Person 1 mainly owns this.

context/

Memory layer.

```
Entire checkpoints
  ↓
context extraction
  ↓
useful previous context
  ↓
Agent
```

Person 2 mainly owns this.

entire/

Everything talking to Entire.

Keep this isolated so rest of code does not depend heavily on Entire internals.

git/

Git-related operations:

```
commits
branches
changed files
history
```

cli/

User-facing commands.

Example:

Bash

```
contextforge init
contextforge start
contextforge context
contextforge checkpoint
```

tests/

Important for hackathon. Need prove:

```
agent event
  ↓
captured
  ↓
Entire checkpoint
  ↓
context retrieved
  ↓
next session receives context
```

One change I recommend

Since this project must live in **Entire's designated fork**, don't assume this exact top-level structure until we inspect that fork.

First run:

Bash

```
git clone <ENTIRE_FORK_URL>
cd <ENTIRE_FORK>
ls
```

Then:

Bash

```
find . -maxdepth 2 -type f | head -100
```

We should **fit ContextForge into Entire's existing architecture**, not create unrelated project sitting beside it.

Next step: **show me the output of `ls` from your Entire fork.** Then we can design exact folders and decide where each teammate works.

[stay in caveman mode — LITE]

tell ai to make this folder stucture

Use this prompt with your coding agent:

 Edit

Create initial folder structure for our hackathon project **ContextForge**.

Important:

- This project is an integration with **Entire**.
- Do NOT build a separate coding agent.
- First inspect existing repository structure and follow its conventions.
- Do not delete, rename, or modify existing Entire files unless required.
- Create only necessary directories/files.
- Use existing project language, package manager, build system, and naming conventions.

Target structure:

```
docs/  
├─ architecture.md  
├─ setup.md  
└─ demo.md
```

```
src/  
├─ adapter/  
│   ├─ index  
│   ├─ events  
│   └─ session  
│  
├─ context/  
│   ├─ index  
│   ├─ collector  
│   └─ formatter
```

```
|   └─ recall
|
└─ entire/
    │   └─ index
    │   └─ checkpoint
    │   └─ client
    │
    └─ git/
        │   └─ index
        │   └─ history
        │
        └─ cli/
            │   └─ index
            │   └─ commands
```

```
tests/
└─ adapter/
└─ context/
└─ entire/
└─ integration/
```

```
examples/
└─ demo-project/
```

```
scripts/
└─ setup
└─ demo
```

Use appropriate file extensions based on existing repository language.

For now:

1. Inspect repository.
2. Identify where this integration should live.
3. Create folder structure.
4. Add minimal placeholder files only where needed.
5. Do not implement functionality yet.
6. Do not invent APIs or Entire interfaces.
7. Report exactly what files/directories were created and why.
8. Do not commit changes.

Use this first. It makes agent inspect Entire before blindly creating folders.

[stay in caveman mode — LITE]

<https://github.com/entireio> so do we have to use from this

Yes. **This is exactly where we should start.**

I checked Entire's GitHub organization. It has several repos, but for our hackathon topic, two matter most:

- ``entireio/cli`` — core Entire CLI. Captures AI sessions, links them to commits, supports checkpoints/rewind.  GitHub
- ``entireio/external-agents`` — specifically made for **external agent integrations**. Entire describes it as standalone agent binaries that extend Entire with support for additional AI coding agents.  GitHub

For our hackathon

Our work should probably happen in:

`entireio/external-agents`

Not build completely separate `ContextForge` from scratch.

Structure becomes more like:

```
entireio/  
└─ external-agents/  
    ├── existing-agent-1/  
    ├── existing-agent-2/  
    ├── ...  
    └─ contextforge/  
        ├── ...  
        └─ ...
```

But **don't create** `contextforge/` **yet.**

First we need inspect `external-agents` and understand how existing integrations are structured.

The repo itself says these are **standalone external agent binaries** extending Entire CLI.

 GitHub

Important correction to our earlier plan

We don't need:

```
ContextForge
├─ entire/
├─ git/
├─ context/
├─ adapter/
└─ ...
```

That could duplicate functionality Entire already provides.

Instead:

```
Existing Agent
    ↓
Our new Entire adapter
    ↓
Entire CLI
    ↓
Checkpoint + Git + Context
```

Much cleaner.

Next move: clone `entireio/external-agents`, inspect its folders, then choose an existing adapter as template. That is where our three-person work should begin.

[stay in caveman mode — LITE]

so do we have to make a web app ?

No. Web app not required.

For this hackathon, core product can be:

```
AI Coding Agent
    ↓
Our Entire Integration
    ↓
Entire CLI
    ↓
Git + Agent Context
```


Could be:

- CLI tool
- Agent plugin/hook
- Editor integration
- CI/workflow integration

No React. No frontend. No backend required unless our chosen workflow needs one.

Best hackathon approach

Build **integration first**, then add small UI only if it makes demo stronger.

Example demo:

```
$ agent

> Add user authentication

Agent changes code
    ↓
Entire captures session
    ↓
Git commit
    ↓
Entire checkpoint
```

Next session:

```
> Continue authentication work
```

Agent retrieves previous development context.

That is enough for strong hackathon demo.

Don't build web app just because hackathon projects often have one. Spend time making context capture/retrieval work reliably.

[stay in caveman mode — LITE]

so how do we start and all

Start from **Entire repo**, not web app.

Step 1 — Clone external-agents

Bash

```
git clone https://github.com/entireio/external-agents.git
cd external-agents
```

Then inspect:

Bash

```
ls
find . -maxdepth 2 -type f
```

Goal: understand how existing agent integrations work.

Step 2 — Pick target agent

Don't pick yet.

Look at existing integrations and choose one agent/workflow that:

- Has accessible hooks/API
- Can expose prompts + tool calls + file changes
- Has enough functionality for a good demo
- Is not already supported by Entire

Possible targets:

```
Aider
OpenHands
Roo Code
Windsurf
Zed Agent
Devin
```

But verify current support before choosing.

Step 3 — Run existing integration

Pick one existing integration as reference.

Run its setup/demo.

Understand:

```
Agent starts
↓
```

```
Agent event generated
  ↓
Integration receives event
  ↓
Entire receives context
  ↓
Checkpoint created
  ↓
Git commit linked
```

Don't code until team understands this flow.

Step 4 — Team splits

Person 1 — Agent integration

```
agent → events → Entire
```

Find how target agent exposes events and build adapter.

Person 2 — Context

```
Entire checkpoint
  ↓
previous context
  ↓
new agent session
```

Figure out how context gets stored/retrieved and what useful context should be passed forward.

Person 3 — Testing + demo

```
setup
tests
sample repo
README
demo script
```

Also test Person 1 + 2 code continuously.

Step 5 — First milestone

Don't aim for full product.

Aim for this:

```
User gives prompt
  ↓
```

```
Agent works
  ↓
Our integration captures it
  ↓
Entire creates checkpoint
  ↓
Git has corresponding change
```

Once that works:

```
New session
  ↓
"Continue previous work"
  ↓
Previous Entire context retrieved
  ↓
Agent understands previous work
```

That is your **MVP**.

Step 6 — Demo magic

Show two sessions.

Session 1

```
"Build authentication system"
```

Agent works → code changes → Entire checkpoint.

Session 2

```
"Continue authentication work. Add password reset."
```

Agent gets previous context.

Then show:

```
Entire checkpoint
├─ original task
├─ agent activity
├─ files changed
├─ Git commit
└─ previous context
```

That makes hackathon value obvious.

Your immediate action

Don't create folders yet.

Run:

Bash

```
git clone https://github.com/entireio/external-agents.git
cd external-agents
ls
```

Send me output.

Then we map **actual Entire architecture** → **our project** → **3-person tasks**.

[stay in caveman mode — LITE]

```
PS C:\CS\CS Projects\external-agents> ls
>> find . -maxdepth 2 -type f
```

Directory: C:\CS\CS Projects\external-agents

Mode	LastWriteTime	Length	Name
d----	06-Sep-26 10:22 AM		.claude
d----	06-Sep-26 10:22 AM		.codex

Show more

Good. This tells us a lot.

`find` failed because you're using **PowerShell**, not Linux shell. No problem.

Most important part:

agents/

This is where we should look first.

Also existing integrations:

```
.claude/  
.codex/  
.cursor/  
.opencode/
```

So do not create our own architecture yet. Entire already has structure for external agents.

Next command

Run:

PowerShell

```
Get-ChildItem .\agents -Directory
```

Then:

PowerShell

```
Get-ChildItem .\agents -Recurse -File | Select-Object FullName
```

Also inspect README:

PowerShell

```
Get-Content .\README.md -TotalCount 200
```

Why?

We need answer three things:

1. How Entire expects external agents to integrate
2. What agents are already supported
3. Where our new agent integration belongs

Then we choose target.

Don't create `src/`, `context/`, `git/`, etc. yet. Entire already has architecture.

Send output of those 3 commands. Then we can map exact work for all 3 teammates.

Pasted text.txt



Good. Output confirms architecture.

Your repo already has exactly what hackathon asks for:

```
agents/
├─ entire-agent-amp
├─ entire-agent-goose
├─ entire-agent-grok
├─ entire-agent-kilo
├─ entire-agent-kiro
├─ entire-agent-omp
└─ entire-agent-qwen
```

README confirms these are **standalone binaries that teach Entire CLI how to work with agents it does not natively support**. They capture transcripts, lifecycle hooks, and checkpoints through Entire's external-agent protocol.

Pasted text

So our project = one new agent

We need pick an agent **not already here**.

For example:

```
agents/
├─ entire-agent-amp/
├─ entire-agent-goose/
├─ ...
└─ entire-agent-<OUR-AGENT>/
```

Inside, existing pattern looks roughly:

```
entire-agent-<name>/
├─ AGENT.md
├─ README.md
├─ go.mod
├─ mise.toml
├─
├─ cmd/
│   └─ entire-agent-<name>/
│       └─ main.go
├─
├─ internal/
│   └─ <agent>/
│       ├── agent.go
│       ├── hooks.go
│       ├── transcript.go
│       └─ ...
```

```
| |
| |└─ protocol/
| |   └─ protocol.go
| |   └─ types.go
| └─ scripts/
```

README also says new agents need:

```
mise.toml
build task
test task
```

so root testing/CI automatically discovers them.

Pasted text

Important: we already have team structure

Now split work based on actual repo.

Person 1 — Agent adapter

Work here:

```
agents/entire-agent-<name>/internal/<name>/
```

Own:

```
agent.go
hooks.go
transcript.go
types.go
```

Main problem:

```
OUR AGENT
  ↓
hooks/events
  ↓
Entire protocol
```

Person 2 — Protocol + tests

Work here:

```
agents/entire-agent-<name>/internal/protocol/
tests/
e2e/
```


Understand protocol and make sure:

```
prompt
tool activity
transcript
session lifecycle
checkpoint
rewind
```

work correctly.

README specifically says testing has **protocol compliance + lifecycle integration + unit tests**.

Pasted text

Person 3 — Setup + docs + demo

Work here:

```
agents/entire-agent-<name>/README.md
agents/entire-agent-<name>/AGENT.md
agents/entire-agent-<name>/mise.toml
scripts/
doc/
```

Make:

```
clone
↓
build
↓
install
↓
entire enable
↓
run agent
↓
checkpoint
↓
rewind
```

easy to demonstrate.

But first: choose agent

This is our **next major decision**.

Already covered:

Amp
Goose
Grok
Kilo
Kiro
Oh My Pi
Qwen Code

So don't choose these.

Pasted text

I recommend we investigate **OpenHands, Aider, Roo Code, Windsurf, or another agent with accessible hooks/events.**

Once we choose, I can give you:

```
exact folder
exact files
exact responsibilities for 3 people
first commands
first coding task
```

No need to invent architecture. **We copy Entire's proven external-agent pattern and build one missing integration.**

[stay in caveman mode — LITE]

bro honestly i dont how we have to build like what and how to work like do we have to work in this repo of the entire or make our own and use in it?

Yes. You should work inside the Entire `external-agents` repo.

Think caveman:

```
ENTIRE REPO
  ↓
you add ONE new agent integration
  ↓
Entire learns how to talk to that agent
```

You **do NOT** build a whole new coding agent.

You **do NOT** make a separate product and just call Entire.

What you actually build

The repo already has:

```
agents/  
├─ entire-agent-amp  
├─ entire-agent-goose  
├─ entire-agent-grok  
├─ entire-agent-kilo  
├─ entire-agent-kiro  
├─ entire-agent-omp  
└─ entire-agent-qwen
```

You add something like:

```
agents/  
└─ entire-agent-aider/  
    ├─ cmd/  
    │   └─ entire-agent-aider/  
    │       └─ main.go  
    ├─ internal/  
    │   ├─ aider/  
    │   │   ├─ agent.go  
    │   │   ├─ hooks.go  
    │   │   └─ transcript.go  
    │   └─ types.go  
    └─ protocol/  
├─ tests...  
├─ README.md  
├─ AGENT.md  
├─ go.mod  
└─ mise.toml
```

The exact agent name is **not decided yet**. We first pick an agent that Entire doesn't already support.

What your code does

Imagine the user runs:

```
aider
```

and says:

```
"Fix the login bug"
```

Aider does its normal coding work.

Your integration watches/translates its events:

```
Aider
  ↓
our entire-agent-aider
  ↓
Entire protocol
  ↓
Entire CLI
  ↓
Git + checkpoint + transcript/context
```

So Entire can understand:

```
What did the agent do?
What files changed?
What did the user ask?
What tools/actions happened?
When did the session start/stop?
Can Entire create a checkpoint?
Can we rewind/resume?
```

The important part

The hackathon wording says the **submitted implementation must live in the designated Entire fork.**

So your workflow should be:

```
GitHub
  ↓
Fork entireio/external-agents
  ↓
Clone YOUR fork
  ↓
Work inside that repo
  ↓
Add your new agent integration
  ↓
Test it
  ↓
Submit your fork/PR
```

Not:

```
Make random project
```

↓

```
somehow call Entire
```

That would likely be too thin and miss the point.

Your 3-person team

I would do:

Person 1 — Agent integration

Build:

```
agents/entire-agent-<name>/internal/<name>/
```

Handles the actual agent → Entire connection.

Person 2 — Testing/integration

Handles:

```
e2e/  
tests/  
protocol tests
```

Proves:

```
prompt → agent → changes → Entire → checkpoint → rewind
```

Person 3 — docs/demo/build

Handles:

```
README.md  
AGENT.md  
mise.toml  
scripts  
demo flow
```

Makes the thing easy to install and impressive to demonstrate.

So where do we start?

Don't code yet.

First we need to answer ONE question:

Which unsupported coding agent should we integrate?

Then I can literally walk you through:

1. create folder
2. create files
3. copy the correct pattern
4. write first code
5. build
6. test
7. connect to Entire
8. run real agent
9. prove checkpoint/rewind
10. prepare hackathon demo

That's the whole game.

[stay in caveman mode — LITE]

so in this repo we have to work and make our own agent like wich reads our prompt ai responses and asve it as context

Yes — **almost**, but one important correction.

You **do make an agent integration**, but you are **not building the AI model itself**.

Think:

```
Existing AI coding agent
  ↓
YOUR ADAPTER
  ↓
Entire
  ↓
stores development context
```

Your adapter's job is to capture things like:

```
User prompt
  ↓
AI response
  ↓
Files changed
```

```
↓
Tools/actions used
↓
Session start/stop
↓
Entire
↓
transcript/context
↓
Git/checkpoint
```

So yes, the idea is:

“Take an existing AI coding agent that Entire doesn’t support yet, connect it to Entire, and make its development activity persist as useful context.”

But don’t build a new ChatGPT/Cursor from scratch.

For example, if we choose **Aider**:

```
User
↓
Aider
↓
"Fix login bug"
↓
Aider changes auth.py
↓
YOUR entire-agent-aider
↓
Entire
↓
checkpoint + transcript + context
```

Then later:

```
New coding session
↓
"Continue the login work"
↓
Entire context
↓
Previous work / reasoning / changes
↓
Agent continues with knowledge
```

That is the core idea.

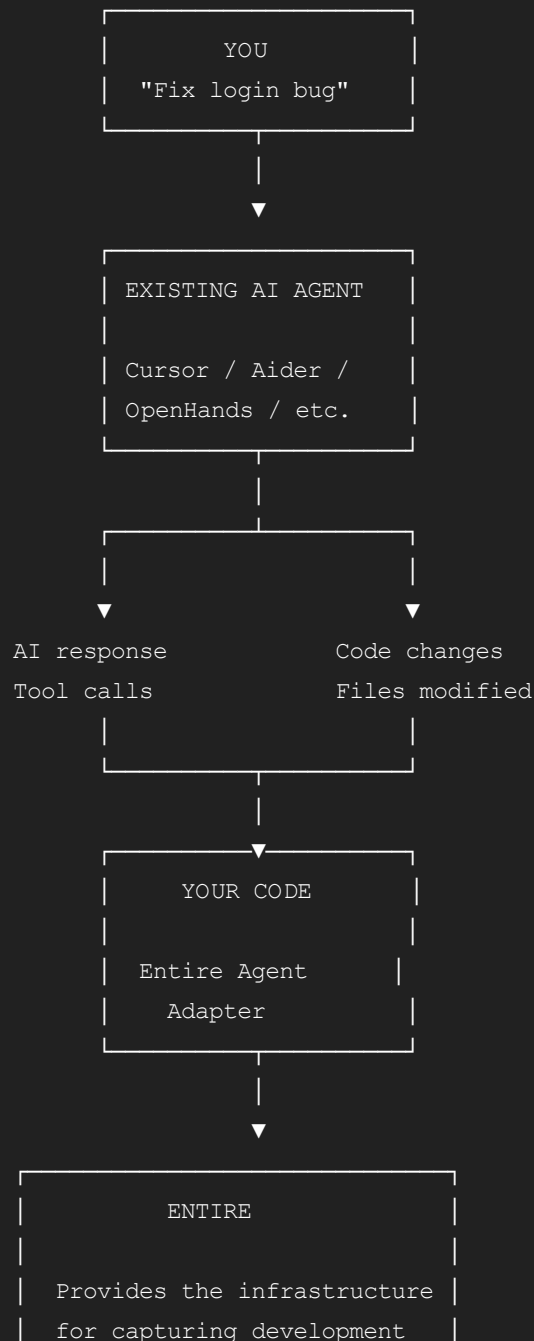
One thing we need to decide before coding: which existing AI agent we will connect to Entire. Then I can show you exactly what code we build and where it goes.

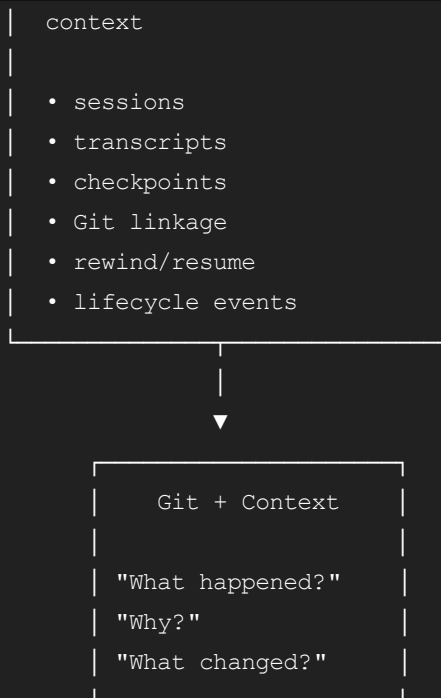
[stay in caveman mode — LITE]

so i am still confused so in a graphica repn explain me and mention what i have to do what entire is providing

Yes. Let's make it visual.

Big picture





What Entire gives you

You do not build all this:

- ✗ Git checkpoint system
- ✗ Git integration
- ✗ Session infrastructure
- ✗ Rewind system
- ✗ Entire protocol
- ✗ Entire CLI
- ✗ Context storage architecture

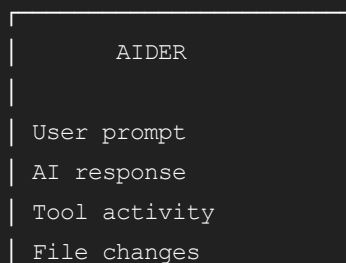
Entire already provides that.

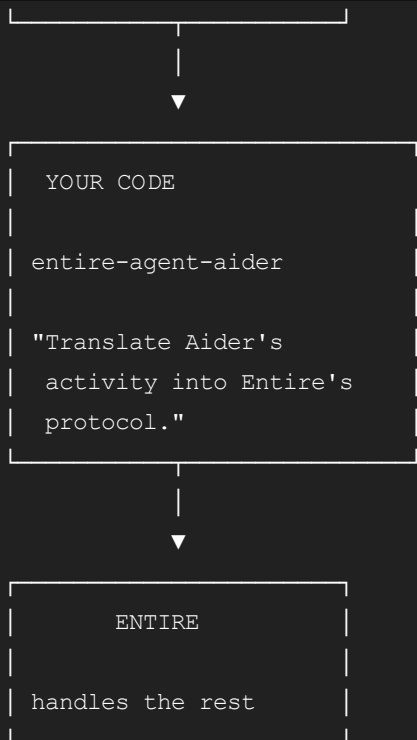
Your job is to make a **new agent understandable by Entire**.

What YOU build

Suppose we choose **Aider**.

You build:





In caveman words

Entire = brain/storage/infrastructure already built.

Your code = translator/adaptor.

You teach Entire:

"When Aider does THIS, it means THIS Entire event."

For example:

```
Aider starts
  ↓
Your adapter
  ↓
Entire: SESSION_STARTED
```

```
User gives prompt
  ↓
Your adapter
  ↓
Entire: capture prompt/transcript
```

```
Aider changes login.py
  ↓
Your adapter / Entire
```

```
↓  
  
Work reaches checkpoint  
↓  
Entire  
↓  
Git checkpoint
```

The repo you are working in

You fork:

```
entireio/external-agents
```

Then add:

```
agents/  
├─ entire-agent-amp  
├─ entire-agent-goose  
├─ entire-agent-grok  
├─ entire-agent-kilo  
├─ entire-agent-kiro  
├─ entire-agent-omp  
├─ entire-agent-qwen  
├─  
└─ entire-agent-YOUR-AGENT ← YOU
```

That new folder is basically your hackathon project.

Your final demo

You want to be able to show:

```
1. Start agent  
↓  
2. Give prompt  
↓  
3. AI works  
↓  
4. Files change  
↓  
5. Entire captures the session  
↓  
6. Entire creates checkpoint  
↓
```

7. Show stored context / transcript

↓

8. Continue or rewind work

That's it.

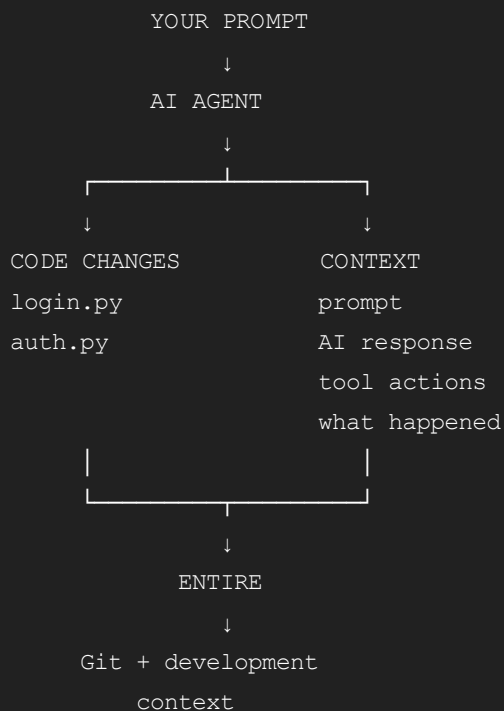
You are not making "another Cursor."

You are making a **bridge** that connects a new AI coding agent to Entire's existing **system**.

[stay in caveman mode — LITE]

so like with code we also push the context

Exactly. But there are **two things** being pushed/recorded:



So when you **commit/push code**, Entire can associate the development context with that code.

Think:

```
Git commit:
"Fixed login bug"
```

+

```
Entire context:
"User asked to fix login bug
  AI investigated auth.py
  AI changed token validation
  AI ran tests
  AI explained why the fix works"
```

↓

ONE HISTORY

That's the important hackathon idea:

Code tells you WHAT changed.

Context tells you WHY and HOW it changed.

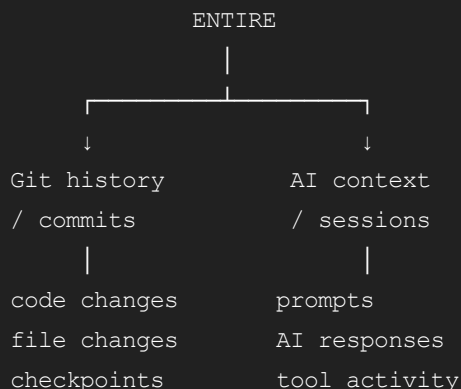
Your adapter's job is to get that context from the unsupported AI agent and feed it into Entire's system.

One correction: **you aren't necessarily directly "pushing context to GitHub."** Entire handles the context/session/checkpoint machinery. Your integration connects the agent's activity to Entire.

[stay in caveman mode — LITE]

this context will go on git or ewentire

Both, but different roles.



So:

- **Git** → stores the **code/history**.

- **Entire** → captures and manages the **AI development context** and connects it to Git commits/checkpoints.

The key relationship is:

```
"Why did this code change?"
```

```
↓
```

```
Entire
```

```
↓
```

```
Git commit
```

```
↓
```

```
"This context belongs  
to this development"
```

So don't think **"we push the AI context to GitHub."**

Think:

Git stores the code. Entire preserves the story behind the code.

ChatGPT can make mistakes. Check important info.